

1) Identified core classes

User

- id: int
- firstName: string
- lastName: string
- email: string
- roleId: int
- methods: getFullName(), getID()

Role

- id: int
- name: string
- methods: none

Student (extends User)

- id: int (students.ID)
- userId: int (foreign key to users.ID)
- studentId: string (university student number)
- programmeId: int
- level: int
- status: enum {active, inactive}
- dateOfEnrollment: Date
- methods: registerCourse(), dropCourse(), viewTranscript()

Faculty (extends User)

- id: int (faculty.ID)
- userId: int
- facultyId: string
- departmentId: int
- position: string

- status: enum
- hireDate: Date
- methods: assignGrade(), getCourseAssignments()

Department

- id: int
- name: string
- methods: listProgrammes()

Programme

- id: int
- departmentId: int
- name: string
- programmeType: enum {BSc,BA,MSc,MA,PhD}
- methods: listCourses()

Course

- id: int
- programmeId: int
- level: int
- semester: string
- code: string
- title: string
- credit: int
- methods: getPrerequisites()

Prerequisite

- id: int
- courseId: int
- prerequisiteCourseId: int

CourseAssignment (which faculty teaches which course)

- id: int
- facultyId: int
- courseId: int
- semester: string
- academicYear: string
- methods: none or isAssigned()

StudentCourse

- id: int
- studentId: int
- courseId: int
- semester: string
- academicYear: string
- registrationDate: timestamp
- status: enum {Registered, Dropped, Completed}
- methods: markCompleted()

Grade

- id: int
- studentId: int
- courseId: int
- facultyId: int
- grade: string (A, B+, etc.)
- gradePoints: decimal
- semester: string
- academicYear: string
- dateEntered: timestamp
- methods: computeGPAComponent()

2) Relationships (explanation + multiplicities)

1. User — Role

- a. Relationship: User $\text{roleId} \rightarrow \text{Role}$
- b. Multiplicity: User (*) $\rightarrow$ 1 Role (1)
- c. Type: Association (Role is independent).
- d. Note: Enforce RBAC via Role.

2. Student — User

- a. Relationship: Student has a foreign key to User.
- b. Multiplicity: Student 1 $\rightarrow$ 1 User
- c. Type: Composition-like (student depends on user account).

3. Faculty — User

Faculty 1 $\rightarrow$ 1 User

Type: Composition-like.

4. Programme — Department

- a. Programme * $\rightarrow$ 1 Department (One department can run many programmes).
- b. Type: Aggregation.

5. Course — Programme

- a. Course * $\rightarrow$ 1 Programme (many courses belong to a programme).
- b. Type: Aggregation.

6. Prerequisites

- a. Course * $\leftrightarrow$ * Course (self-referencing many-to-many implemented via prerequisites).
- b. Type: Association (junction table).

7. CourseAssignment (Faculty $\leftrightarrow$ Course)

- a. Many-to-many resolved: Faculty $*$ $\leftrightarrow$ $*$ Course via course_assignments.
- b. Type: Association (each assignment links faculty to the course for a semester).

8. StudentCourse (Student $\leftrightarrow$ Course)

- a. Many-to-many resolved: Student $*$ $\leftrightarrow$ $*$ Course via student_courses.
- b. Type: Association (with attributes like registrationDate and Status).

9. Grades

- a. Each Grade links Student (1) + Course (1) + Faculty (1) — a ternary relation modeled as record.
- b. Multiplicity: Student $1 \rightarrow *$ Grades, Course $1 \rightarrow *$ Grades, Faculty $1 \rightarrow *$ Grades.

10. Users and constraints

- Deleting a User is blocked depending on policy.
- Course deletion while students enrolled is prevented.